warehouse-rl

Документация проекта и исходного кода

Исследовательский фреймворк для изучения *обобщения*
RL-политик распределения задач между мобильными роботами
на ранее не встречавшихся планировках склада

Пакет:	`warehouse-rl` (MIT)
Версия:	0.1.0
Python:	$\geq$ 3.10 (разработка на 3.14)
Статус:	Итерация 1 — симулятор, A*, greedy-базлайн, метрики
Дата отчёта:	11 октября 2026 г.

Документ сгенерирован автоматически на основе анализа содержимого репозитория `RRS`.

Содержание

1 Введение и исследовательская задача **2**
1.1 Модель задачи и допущения . . . 2
1.2 Почему собственный симулятор . . . 2

2 Структура репозитория **3**
2.1 Назначение ключевых файлов . . . 3

3 Архитектура и поток данных **4**

4 Система конфигурации **4**
4.1 Иерархия dataclass-ов . . . 4
4.2 Наследование и переопределения . . . 5

5 Модель среды **5**
5.1 Планировка: `Layout` и генератор . . . 6
5.2 Заказы . . . 6
5.3 Робот . . . 7
5.4 Симуляция: `WarehouseSimulation` . . . 7
5.5 Рендеринг . . . 8

6 Навигация: A* и оракул расстояний **9**

7 Алгоритмы распределения задач **10**
7.1 Жадный базлайн (`greedy.py`) . . . 10
7.2 Заглушки PPO и DQN . . . 10

8 Метрики **10**

9 Экспериментальные сценарии **11**
9.1 Обучение (`experiments/train.py`) . . . 11
9.2 Оценка (`experiments/evaluate.py`) . . . 12
9.3 Сравнение (`experiments/compare.py`) . . . 12

10 CLI-скрипты **12**

11 Тесты **13**

12 Конфигурационные файлы и артефакты **13**
12.1 Конфигурации . . . 13
12.2 Артефакты результатов . . . 13

13 Результаты базовой оценки (greedy) **14**
13.1 Временные метрики по планировкам . . . 14
13.2 Производительность и загрузка . . . 15
13.3 Разброс и корреляции . . . 15

14 Замечания по коду и наблюдения **16**

15 Ограничения, допущения и дорожная карта **17**
15.1 Сознательные упрощения . . . 17
15.2 Репродуктивность и научная честность . . . 17
15.3 Дорожная карта . . . 17

16 Установка и запуск **17**

17 Краткий API-справочник **18**

1 Введение и исследовательская задача

Проект **warehouse-rl** — это исследовательский фреймворк, отвечающий на вопрос:

Исследовательский вопрос

Может ли политика обучения с подкреплением, обученная на ограниченном числе планировок склада, эффективно решать задачи на *ранее не виденных* планировках?

В отличие от работ, нацеленных на пиковую производительность в одной среде, данный проект изучает **обобщение между средами**. Задача обучения — **распределение задач** (task allocation): назначение поступающих заказов однородным мобильным роботам. Низкоуровневая навигация выполняется классическим алгоритмом A* и *не* изучается RL. Политика лишь решает, *какой робот какой заказ обслуживает*.

1.1 Модель задачи и допущения

Аспект	**Допущение**
Роботы	Фиксированное число *однородных* роботов (одинаковая скорость и грузоподъёмность).
Склад	Сетка: стеллажи/препятствия (`#`), пункты подбора (`P`), пункты выгрузки (`D`), точки старта роботов (`R`).
Заказы	Динамически появляющиеся задания «забрать–доставить» (`P` → `D`).
Решения	Политика назначает ожидающие заказы свободным роботам каждые `decision_interval` шагов.
Навигация	A* по статической сетке (4- или 8-связность); движение детерминировано; столкновения между роботами не моделируются.
Обучение	RL управляет *только распределением*; он не выдаёт пошаговые перемещения.
Обобщение	Политики оцениваются на планировках, *не встречавшихся* при обучении.

Робот подчиняется конечному автомату idle → to_pickup → picking → to_dropoff → dropping → idle. Заказы хранят временные метки, поэтому времена ожидания, обслуживания и завершения вычисляются точно.

1.2 Почему собственный симулятор

Известный бенчмарк **RWARE** (`multi-robot-warehouse`, NeurIPS 2021) был рассмотрен, но не принят, поскольку:

1. RL управляет *низкоуровневым движением* (`TURN_LEFT/RIGHT/FORWARD/LOAD`) — нет внешнего слоя распределения задач и нет классического A*-планировщика.
2. Заказы — это запросы на доставку *стеллажа*, а не модульные задания «забрать–доставить» с измеримыми временами ожидания/завершения.
3. Обучающие планировки фиксированы (`tiny/small/medium/large`), что затрудняет чистое исследование обобщения по планировкам.

Собственный компактный симулятор даёт полный контроль над планировками, потоком заказов, метками времени и метриками, а планировщик A* остаётся *фиксированным* решателем нижнего уровня.

2 Структура репозитория

```
RRS/├──
 README.md                    # описание проекта и инструкции├──
 pyproject.toml               # метаданные, зависимости, конфигурация инструментов├──
 configs/|
   ├── default.yaml             # базовая конфигурация|
   ├── training.yaml            # extends default.yaml; алгоритм и параметры обучения|
   └── evaluation.yaml          # extends default.yaml; параметры оценки и сравнения├──
 src/warehouse_rl/|
   ├── __init__.py              # публичный API верхнего уровня|
   ├── config.py                # типизированная конфигурация на YAML|
   ├── environment/|
   |    ├── warehouse.py          # ядро: дискретная пошаговая симуляция|
   |    ├── robot.py              # конечный автомат однородного робота|
   |    ├── orders.py             # модель заказа + стохастический поток заказов|
   |    ├── layout_generator.py   # представление планировки + процедурная генерация|
   |    └── rendering.py          # ASCII/PNG/GIFвизуализация– опционально()|
   ├── navigation/|
   |    └── astar.py              # A* + кэширующий оракул расстояний|
   ├── algorithms/|
   |    ├── greedy.py             # жадный базлайн реализован()|
   |    ├── ppo.py                # заглушка веха( M4)|
   |    └── dqn.py                # заглушка веха( M6, опционально)|
   ├── experiments/|
   |    ├── train.py              # запуск обучения (greedy –> no–op; PPO/DQN позже)|
   |    ├── evaluate.py           # не зависящая от политики оценка –> CSV|
   |    └── compare.py            # сравнение алгоритмов на одних планировкахсидах/|
   └── metrics/|
        └── metrics.py            # эпизодные метрики + агрегация├──
 layouts/|
   ├── train/                   # 10 сгенерированных обучающих планировок|
   └── test/                    # 10 тестовых планировок непересекающиеся( сиды)├──
 tests/                       # pytest: навигация, планировки, среда, конфиг├──
 scripts/|
   ├── generate_layouts.py      # генерация наборов планировок|
   ├── train.py                 # CLI запуска обучения|
   ├── evaluate.py              # CLI оценки на тестовых планировках|
   └── demo.py                  # сквозное smokедемо– с рендерингом├──
 results/                     # CSV, снимки, артефакты запусков└──
 notebooks/                   # аналитические ноутбуки запланированы()
```

2.1 Назначение ключевых файлов

Файл	Роль
`src/warehouse_rl/config.py`	Типизированные dataclass-конфиги, `extends`-наследование YAML, dotted-overrides.
`.../environment/warehouse.py`	`WarehouseSimulation` — ядро симуляции, не зависит от Gymnasium.
`.../environment/robot.py`	`Robot` — состояние и движение одного робота.

Файл	Роль
`.../environment/orders.py`	`Order`, `OrderGenerator` — модель заказа и поток Бернулли.
`.../environment/layout_generator.py`	`Layout`, генерация и проверка связности планировок.
`.../environment/rendering.py`	`save_snapshot`, `animate_episode` (Matplotlib, ленивый импорт).
`.../navigation/astar.py`	`astar`, `DistanceOracle` — планирование и кэш расстояний.
`.../algorithms/greedy.py`	`GreedyPolicy`, `make_policy` — нестатистический базлайн.
`.../algorithms/ppo.py, dqn.py`	Заглушки с контрактом интерфейса.
`.../experiments/*.py`	Раннеры обучения, оценки и сравнения.
`.../metrics/metrics.py`	`EpisodeMetrics`, вычисление и выгрузка метрик.
`scripts/*.py`	Тонкие CLI-обёртки над библиотекой.
`tests/*.py`	40 тестов pytest.

3 Архитектура и поток данных

Проект разделён на слои с односторонними зависимостями: `config` → `environment` → `navigation` → `algorithms` → `experiments` → `metrics`. Ядро симуляции (`environment`) намеренно *не* зависит от Gymnasium или PyTorch — это позволяет тестировать его изолированно, а RL-обёртку (веха M3) сделать тонкой.

```
configs/*.yaml ── load_config() ──> ExperimentConfig / TrainingConfig / EvalConfig
                                            |
layouts/*.txt ── Layout.read() ──> WarehouseSimulation <── policy.assign(sim)
                                            |
                      step(): arrivals -> allocation -> dynamics -> timeouts
                                            |
                                      metrics ──> CSV / summary
```

Ключевой интерфейс расширения — метод `assign(sim)`: любой объект с этим методом (и, опционально, атрибутом `name`) может быть вычислен одним и тем же кодом оценки. Именно это делает `experiments/evaluate.py` независимым от политики.

4 Система конфигурации

Модуль `config.py` — единый источник истины для всех настраиваемых параметров, что критично для воспроизводимости (сиды, число роботов, наборы планировок). Конфигурации хранятся как вложенные словари YAML и разбираются в dataclass-ы.

4.1 Иерархия dataclass-ов

```python
@dataclass
class ExperimentConfig:
    seed: int = 0
    warehouse: WarehouseConfig = field(default_factory=WarehouseConfig)
    orders:    OrderConfig     = field(default_factory=OrderConfig)
    layout:    LayoutConfig    = field(default_factory=LayoutConfig)
```

```
    greedy:    GreedyConfig    = field(default_factory=GreedyConfig)
    reward:    RewardConfig    = field(default_factory=RewardConfig)

@dataclass
class TrainingConfig(ExperimentConfig):
    algorithm: str = "greedy"        # "greedy" | "ppo" | "dqn"
    n_training_layouts: int = 1
    seeds: List[int] = field(default_factory=lambda: [0])
    total_timesteps: int = 100_000
    output_dir: str = "results/training"

@dataclass
class EvaluationConfig(ExperimentConfig):
    algorithms: List[str] = field(default_factory=lambda: ["greedy"])
    n_episodes: int = 5
    seeds: List[int] = field(default_factory=lambda: [0, 1, 2])
    output_dir: str = "results/evaluation"
    save_csv: bool = True
```

Секции `warehouse`, `orders`, `layout`, `greedy`, `reward` представлены отдельными dataclass-ами и перечислены в словаре `_SECTION_TYPES`. Значения кэшируются через `get_type_hints`, что позволяет лучшим образом приводить типы.

4.2 Наследование и переопределения

Загрузка поддерживает три механизма:

1. **extends** — ключ верхнего уровня `extends: base.yaml` подгружает родительский файл и рекурсивно сливает его с текущим (`deep_merge`);
2. **dotted-overrides** — переопределение по точечному пути, например `{"warehouse.n_robots": 6}`;
3. **толерантность** — незнакомые ключи не ломают загрузку, а выводят предупреждение в консоль.

```
def deep_merge(base, override):
    result = copy.deepcopy(base)
    for key, value in override.items():
        if key in result and isinstance(result[key], dict) and isinstance(value, Mapping):
            result[key] = deep_merge(result[key], value)
        else:
            result[key] = copy.deepcopy(value)
    return result
```

Загрузка с защитой от циклических зависимостей и сохранение снимка конфигурации для воспроизводимости:

```
def load_config(path=None, *, config_cls=ExperimentConfig, overrides=None):
    data = {}
    if path is not None:
        data = _load_yaml_with_extends(Path(path))   # резолвит extends, ловит циклы
    if overrides:
        data = apply_overrides(data, overrides)      # dottedключи–
    return _build_config(config_cls, data)

def save_config(config, path):
    payload = asdict(config) if is_dataclass(config) else config
    yaml.safe_dump(payload, fh, sort_keys=False)
```

5 Модель среды

5.1 Планировка: `Layout` и генератор

Планировка — это прямоугольная ASCII-сетка. Легенда символов приведена ниже.

Символ	Значение
`.`	свободный пол (проходим)
`#`	препятствие / стеллаж (непроходим)
`P`	пункт подбора заказа (проходим)
`D`	пункт выгрузки/упаковки (проходим)
`R`	точка старта робота (проходим)

Класс `Layout` в `layout_generator.py` валидирует прямоугольность и допустимость символов, а также предоставляет геометрические и семантические аксессоры: `width`, `height`, `walkable_mask()`, `cells(char)`, `pick_locations`, `dropoff_locations`, `robot_starts`, `free_cells`. Чтение и запись — через `Layout.read`/`Layout.write`. Проверки связности реализованы BFS-обходом (`reachable_from`, `is_connected`).

Генерация (`generate_layout`) использует **rejection sampling**:

1. строится рамка из стен (`#`);
2. `_carve_racks` вырезает горизонтальные сегменты стеллажей, оставляя вертикальные проходы;
3. `_place_pick_stations` расставляет `P` рядом со стеллажами;
4. `_place_dropoffs_and_starts` размещает `D` и `R` вдоль служебного коридора;
5. кандидат проверяется на наличие `P`, `D`, `R` и полную связность; при неудаче извлекается новая выборка (до `max_attempts`=25).

```
def generate_train_test_suites(*, n_train, n_test, train_dir, test_dir,
                               generation_seed=20240101, ...):
    train = generate_layout_suite(n_train, seed=generation_seed, prefix="train", ...)
    test  = generate_layout_suite(n_test,  seed=generation_seed + 1_000_003,
                                  prefix="test", ...)
    write_suite(train, train_dir)
    write_suite(test, test_dir)
    return train, test
```

Важно: обучающий и тестовый наборы порождаются *разными* потоками сидов (сдвиг $+1{,}000{,}003$), поэтому тестовые планировки гарантированно не встречались при обучении. В репозитории уже присутствуют по 10 планировок в каталогах `layouts/train` и `layouts/test`.

5.2 Заказы

Модуль `orders.py` описывает задание «забрать–доставить» и его поток.

```
@dataclass
class Order:
    id: int
    pickup: Cell
    dropoff: Cell
    created_step: int
    status: str = PENDING          # pending|assigned|picked|completed|timeout
    assigned_step: Optional[int] = None
    picked_step: Optional[int] = None
    completed_step: Optional[int] = None
    robot_id: Optional[int] = None

    @property
    def waiting_time(self):        # assigned - created
```

```
        return None if self.assigned_step is None else self.assigned_step - self.created_step

    @property
    def completion_time(self):     # completed - created (lead time)
        return None if self.completed_step is None else self.completed_step - self.created_step

    @property
    def service_time(self):        # completed - assigned
        ...
```

`OrderGenerator` реализует поток Бернулли: на каждом шаге с вероятностью `arrival_rate` рождается заказ (случайные координаты подбора и выгрузки). Учитываются ограничения `max_pending` (число ожидающих) и `max_orders` (общий лимит). Источник случайности — `numpy.random.default_rng(seed)`.

5.3 Робот

`robot.py` описывает однородного робота как конечный автомат. Статусы — строки (`idle`, `to_pickup`, `picking`, `to_dropoff`, `dropping`) для удобства логирования и экспорта в CSV. Хранится текущий путь (`path`) и курсор (`path_index`), таймер сервиса (`timer`) и кумулятивная статистика (`distance_travelled`, `busy_steps`, `orders_completed`).

```
def step_along_path(self, *, connectivity=4) -> bool:
    """Один шаг по сетке. True, если путь исчерпан."""
    if self.goal_reached:
        return True
    prev = self.path[self.path_index]
    self.path_index += 1
    nxt = self.path[self.path_index]
    diag = abs(nxt[0] - prev[0]) and abs(nxt[1] - prev[1])
    self.distance_travelled += (2.0 ** 0.5) if (diag and connectivity == 8) else 1.0
    self.position = nxt
    return self.goal_reached
```

5.4 Симуляция: `WarehouseSimulation`

Это ядро проекта. Каждый вызов `step()` продвигает мир на один шаг по следующей схеме.

Фаза	Действие
1	Приход новых заказов (поток Бернулли).
2	Опрос политики каждые `decision_interval` шагов → список пар `(robot_id, order_id)`.
3	Движение роботов вдоль A*-путей и обработка таймеров сервиса.
4	Истечение срока ожидающих (неназначенных) заказов.
5	Учёт статистики (занятость, пройденная дистанция).

```
def step(self, policy=None, assignments=None) -> StepResult:
    if self.step_count ≥ self.config.warehouse.max_steps:
        raise RuntimeError("Episode already finished; call reset() first")
    self.step_count += 1
    result = StepResult(step=self.step_count)

    new_orders = gen.maybe_generate(step, pending_count=len(self.pending_orders()))
    for order in new_orders:
        self.orders[order.id] = order
        result.arrived_order_ids.append(order.id)

```

```
    interval = max(1, int(self.config.warehouse.decision_interval))
    chosen = []
    if assignments is not None:
        chosen = assignments
    elif policy is not None and step % interval == 0:
        chosen = policy.assign(self)
    result.assignments = self._apply_assignments(chosen)

    for robot in self.robots:
        self._advance_robot(robot, result)
    self._handle_timeouts(step, result)
    ...
    return result
```

Назначения **валидируются**: робот должен быть свободен, заказ — в статусе `pending`, а пункт подбора — достижим (`can_assign`). Метод `assign` строит путь и переводит робота в `to_pickup`. Метод `run(policy)` прогоняет эпизод до конца горизонта, возвращая историю `StepResult`.

Внутренняя динамика реализована в `_advance_robot`: движение по пути с учётом `speed` и `connectivity`, обработка фаз `picking`/`dropping` через таймеры, переход к следующему участку (`_start_dropoff_leg`) и завершение (`_complete_order`). Если второй участок недостижим (что не должно происходить в связных планировках), заказ помечается `failed`.

5.5 Рендеринг

`rendering.py` содержит `save_snapshot` (PNG) и `animate_episode` (GIF). Matplotlib импортируется *лениво*, поэтому ядро симулятора не имеет зависимости от графики. ASCII-рендер доступен прямо в симуляторе: `WarehouseSimulation.ascii_render()` накладывает позиции роботов на сетку.

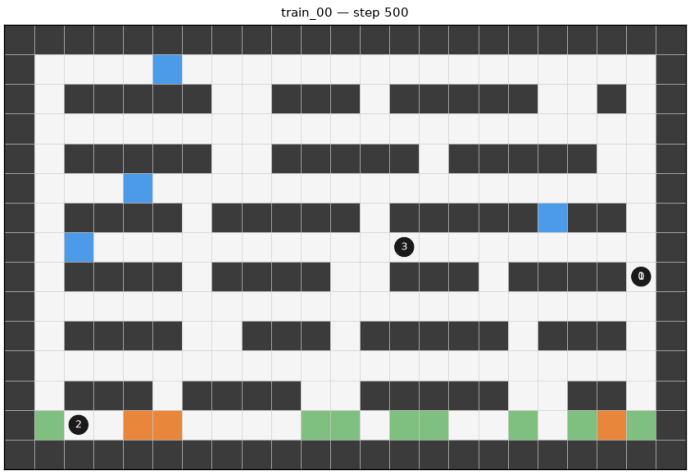


Рис. 1: Снимок состояния симуляции, полученный `save_snapshot()` (файл `results/demo_snapshot.png`): тёмные окружности — роботы, подписанные своими номерами.



Рис. 2: Примеры сгенерированных планировок: обучающая `train_00` и тестовая `test_00`. Символы: `#` — стеллаж, `.` — пол, `P` — подбор, `D` — выгрузка, `R` — старт робота. Каждая планировка проверяется генератором на полную связность.

6 Навигация: A* и оракул расстояний

Модуль `navigation/astar.py` решает классическую подзадачу движения по статической сетке. Используется A* с допустимой эвристикой: манхэттенское расстояние для 4-связности и октильное для 8-связности.

```
def _heuristic(a, b, connectivity):
    dr, dc = abs(a[0]-b[0]), abs(a[1]-b[1])
    if connectivity == 4:
        return float(dr + dc)
    return float(max(dr, dc) + (2.0**0.5 - 1.0) * min(dr, dc))   # октиль

def astar(walkable, start, goal, *, connectivity=4):
    """Минимальный по стоимости путь start..goal включительно() или None."""
    ...
    return _reconstruct(came_from, current)
```

`DistanceOracle` кэширует пути и расстояния, поскольку планировка в течение эпизода не меняется. Это делает жадный базлайн (который на каждом шаге оценивает много пар робот–заказ) дешёвым. Расстояния симметричны, поэтому один результат заполняет оба направления кэша.

```
class DistanceOracle:
    def path(self, start, goal):
        key = (start, goal)
        if key not in self._path_cache:
            self._path_cache[key] = astar(self.walkable, start, goal,
                                          connectivity=self.connectivity)
        return self._path_cache[key]

    def distance(self, start, goal):
        ...
        self._dist_cache[reverse] = value   # симметрия
        return value

    def reachable(self, start, goal):
        return self.distance(start, goal) is not None
```

7 Алгоритмы распределения задач

7.1 Жадный базлайн (`greedy.py`)

Жадная политика — классическая неалгоритмическая точка отсчёта. Реализованы две стратегии.

Стратегия	Поведение
`min_completion_time` (по умолчанию)	Заказы обрабатываются от старых к новым; каждому назначается свободный робот с наименьшим оценённым временем завершения.
`nearest_robot`	Каждому свободному роботу назначается ожидающий заказ с ближайшим пунктом подбора.

```python
def estimate_completion_time(self, sim, robot, order):
    to_pickup = sim.oracle.distance(robot.position, order.pickup)
    if to_pickup is None:
        return None
    to_dropoff = sim.oracle.distance(order.pickup, order.dropoff)
    if to_dropoff is None:
        return None
    wcfg = sim.config.warehouse
    return to_pickup + wcfg.pick_dwell_steps + to_dropoff + wcfg.drop_dwell_steps

def _assign_min_completion_time(self, sim):
    pending = sim.pending_orders()          # от старых к новым
    available = {r.id for r in sim.idle_robots()}
    assignments = []
    for order in pending:
        best_robot, best_cost = None, float("inf")
        for robot_id in sorted(available):
            cost = self.estimate_completion_time(sim, sim.robots[robot_id], order)
            if cost is not None and cost < best_cost:
                best_cost, best_robot = cost, robot_id
        if best_robot is not None:
            assignments.append((best_robot, order.id))
            available.discard(best_robot)
    return assignments
```

Базлайн *детерминирован* и, по замыслу, **никогда не настраивается**. Авторы проекта не предполагают, что RL обязательно его превзойдёт.

7.2 Заглушки PPO и DQN

Файлы `ppo.py` и `dqn.py` определяют *контракт интерфейса* будущих политик (`assign`, `save`, `load`, атрибут `name`), но конструкторы намеренно бросают `NotImplementedError`. Это делает незавершённость явной вместо имитации обучения. Фабрика `make_policy(name, config)` возвращает `GreedyPolicy` для `"greedy"` и бросает `NotImplementedError` для `"ppo"`/`"dqn"`.

8 Метрики

Модуль `metrics/metrics.py` собирает метрики на эпизод и агрегирует их. Все времена — в шагах симуляции, расстояния — в единицах сетки.

Метрика	Определение
Avg completion time	среднее `completed - created`
Avg waiting time	среднее `assigned - created`
Avg service time	среднее `completed - assigned`
Total travel distance	суммарное перемещение роботов (единицы сетки)
Throughput per 100 steps	завершённых заказов на 100 шагов
Robot utilization	занятые робото-шаги / (`n_robots` × длительность)
`episode_duration`	длительность эпизода в шагах
`n_orders_created`	создано заказов
`n_orders_completed`	завершено заказов
`n_orders_timed_out`	истекло по таймауту
`n_orders_failed`	неуспешных (недостижимый участок)

```python
duration = int(sim.step_count)
n_completed = len(completed)
robot_steps = max(1, sim.n_robots * duration)

return EpisodeMetrics(
    layout=..., policy=..., seed=...,
    episode_duration=duration,
    n_orders_created=len(sim.orders),
    n_orders_completed=n_completed,
    avg_completion_time=_safe_mean(completion_times),
    avg_waiting_time=_safe_mean(waiting_times),
    avg_service_time=_safe_mean(service_times),
    total_travel_distance=float(sim.total_travel_distance),
    throughput_per_100_steps=(100.0 * n_completed / duration) if duration else 0.0,
    robot_utilization=float(sim.robot_busy_steps) / robot_steps,
)
```

Функции `records_to_rows`, `save_records_csv` пишут результаты в CSV (через pandas, с запасным вариантом на стандартный `csv`). `aggregate(records)` группирует метрики по `(policy, layout)` и вычисляет `mean`/`std`.

9 Экспериментальные сценарии

9.1 Обучение (`experiments/train.py`)

На текущей вехе обучение отсутствует: `train()` загружает обучающие планировки, создаёт каталог запуска, сохраняет снимок конфигурации (`config.yaml`) и пишет `MANIFEST.txt`. Для greedy это «no-op»-обучение, а для `ppo`/`dqn` поднимается `NotImplementedError` (каталог и снимок конфига при этом уже созданы).

```python
def train(config, *, run_dir=None):
    layouts = load_training_layouts(config)
    run_dir = Path(run_dir) if run_dir else Path(config.output_dir) / config.algorithm
    run_dir.mkdir(parents=True, exist_ok=True)
    save_config(config, run_dir / "config.yaml")

    if config.algorithm == "greedy":
        policy = GreedyPolicy(config.greedy)
        (run_dir / "MANIFEST.txt").write_text(...)
        return TrainingResult(algorithm="greedy", policy=policy, ...)
    if config.algorithm in ("ppo", "dqn"):
        raise NotImplementedError(...)
```

```
    raise ValueError(f"Unknown algorithm '{config.algorithm}'")
```

9.2 Оценка (`experiments/evaluate.py`)

Раннер оценки *не зависит* от алгоритма. Эпизоды прогоняются по всем планировкам, базовым сидам и повторам; для воспроизводимости и независимости итоговый сид вычисляется как `base_seed * 1_000_003 + episode`.

```
def evaluate_policy(policy, layouts, config, *, n_episodes=None, seeds=None, policy_name=None):
    n_episodes = n_episodes or config.n_episodes
    seeds = list(seeds) if seeds is not None else list(config.seeds)
    records = []
    for layout in layouts:
        for base_seed in seeds:
            for episode in range(n_episodes):
                ep_seed = int(base_seed) * SEED_STRIDE + episode
                records.append(run_episode(layout, config, policy,
                                           seed=ep_seed, policy_name=policy_name))
    return records
```

9.3 Сравнение (`experiments/compare.py`)

`compare_algorithms` прогоняет несколько политик на *одних и тех же* тестовых планировках и сидах, пишет `episodes.csv` и `summary.csv`, а `summarize_records` формирует сгруппированную сводку (`mean`/`std` по политике). Это фундамент Эксперимента C.

10 CLI-скрипты

Все скрипты добавляют `src` в `sys.path`, поэтому работают без установки пакета.

Скрипт	Назначение
`generate_layouts.py`	Генерация обучающих и тестовых наборов планировок (по умолчанию 10+10). Флаг `--force` для перегенерации.
`train.py`	Запуск «обучения» (для greedy — создание артефактов запуска).
`evaluate.py`	Оценка политики на тестовых планировках, вывод сводки, запись CSV.
`demo.py`	Сквозное демо: один эпизод greedy, ASCII-кадры, PNG-снимок, опционально GIF.

```
# 1) Сгенерировать планировки
python scripts/generate_layouts.py
# 2) Демо: один greedyэпизод– + снимок
python scripts/demo.py ––every 100 ––snapshot results/demo.png
# 3) Оценка greedy на тестовых планировках
python scripts/evaluate.py ––episodes 3
# 4) Обучение<<>> (greedy – no–op)
python scripts/train.py
# 5) Тесты
python –m pytest
```

11 Тесты

Тестовый набор содержит **40 тестов** pytest (`tests/`), покрывающих четыре области.

Файл	Тестов	Проверяемое
`test_config.py`	8	Загрузка конфигов, extends, dotted-overrides, ошибки типов, save/reload.
`test_layouts.py`	12	Валидация планировок, детерминизм генерации, связность, непересекаемость train/test.
`test_environment.py`	11	Сквозные эпизоды, детерминизм, reset, валидация назначений, таймауты, метрики.
`test_navigation.py`	9	A*: кратчайшие пути, обход препятствий, 8-связность, кэш и симметрия оракула.

Тесты подтверждают ключевые инварианты: политика назначает только свободных роботов на ожидающие заказы; один робот получает не более одного заказа за шаг; эпизод детерминирован при одинаковом сиде; при недостижимом пункте подбора заказ пропускается.

12 Конфигурационные файлы и артефакты

12.1 Конфигурации

`default.yaml` задаёт базовые параметры: размеры сетки, число роботов, времена сервиса, горизонт эпизода, скорость, связность, интервал решений, поток заказов, размеры наборов планировок, стратегию greedy и заглушку reward. `training.yaml` и `evaluation.yaml` подключают его через `extends: default.yaml` и добавляют специфичные поля.

Параметр	Значение по умолчанию
`warehouse.n_robots`	4
`warehouse.max_steps`	500
`warehouse.connectivity`	4
`warehouse.decision_interval`	1
`orders.arrival_rate`	0.35
`orders.max_pending`	25
`layout.n_train / n_test`	10 / 10
`layout.generation_seed`	20240101
`greedy.strategy`	min_completion_time

12.2 Артефакты результатов

- `results/evaluation/greedy_episodes.csv` — 60 эпизодных записей (10 тестовых планировок × 3 базовых сида × 2? — фактически 6 запусков на планировку).
- `results/evaluation/` — каталог для CSV оценки.
- `results/training/greedy/` — снимок конфигурации (`config.yaml`) и `MANIFEST.txt` для запуска greedy.

- `results/demo_snapshot.png` — иллюстративный снимок симуляции.

13 Результаты базовой оценки (greedy)

По имеющемуся CSV (`greedy_episodes.csv`) вычислены средние значения по 60 эпизодам (10 тестовых планировок, 6 повторов каждая). Все планировки завершались за фиксированный горизонт 500 шагов.

Метрика (по эпизодам)	**Среднее**	**Ст. отклонение**
Avg completion time	137.70	18.48
Avg waiting time	110.57	14.55
Avg service time	27.13	4.13
Total travel distance	1694.55	44.89
Throughput per 100 steps	13.95	2.25
Robot utilization	0.954	0.006
Заказов создано	97.97	10.97
Заказов завершено	69.73	11.24

Всего в наборе завершено 4184 заказа; `n_orders_timed_out` и `n_orders_failed` равны нулю (таймауты по умолчанию отключены, недостижимых участков нет). Утилизация роботов очень высока (≈95%), что указывает на достаточную загрузку при `arrival_rate=0.35` и 4 роботах. Эти числа — точка отсчёта для будущих RL-политик, а не утверждение о превосходстве.

13.1 Временные метрики по планировкам

Рисунок 3 показывает, что порядок величин устойчив: время *ожидания* доминирует над временем *обслуживания* примерно в 4 раза. Это ожидаемо: робот получает заказ не мгновенно после его появления, а по решению политики; большая часть lead time — это пребывание заказа в очереди.

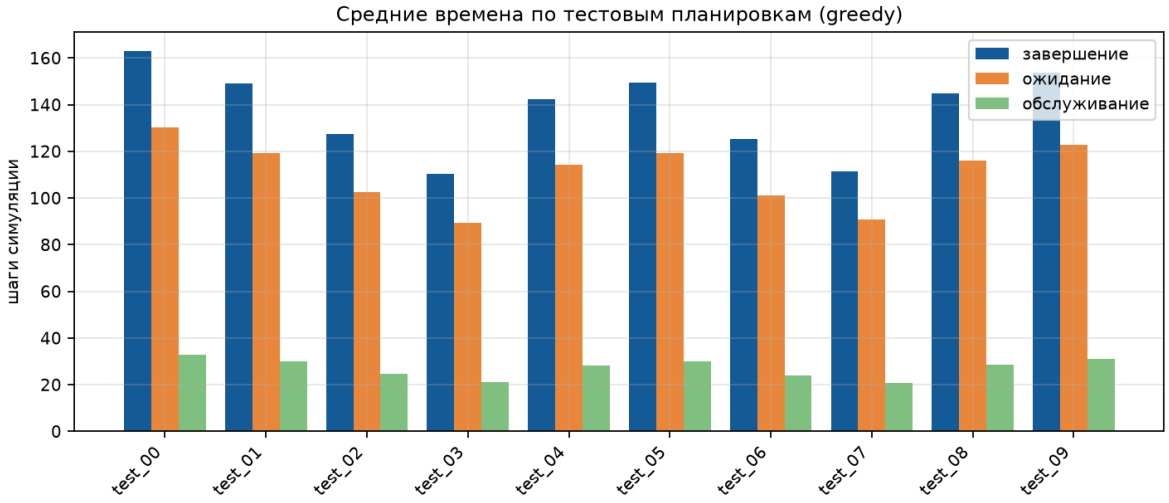


Рис. 3: Средние времена (завершение, ожидание, обслуживание) по каждой из 10 тестовых планировок. Планировки `test_03` и `test_07` дают наименьшее время завершения, `test_00`, `test_05`, `test_09` — наибольшее.

13.2 Производительность и загрузка

На рисунке 4 видна обратная связь производительности с размером времени завершения: планировки с высокой пропускной способностью (`test_03`, `test_07`) имеют и наименьшее время завершения. Утилизация роботов всюду держится в диапазоне 0.94–0.96, то есть роботы почти не простаивают при `arrival_rate=0.35` и 4 роботах.



Рис. 4: Пропускная способность (столбцы, левая ось) и утилизация роботов (линия, правая ось) по тестовым планировкам.

13.3 Разброс и корреляции

Рисунок 5 показывает распределение временных метрик по всем 60 эпизодам: медианы близки к средним, выраженных выбросов нет. Рисунок 6 отображает связь пройденной дистанции и производительности: точки группируются по планировкам, и рост дистанции не приводит к росту пропускной способности — то есть более «далёкие» планировки просто тратят больше перемещений на тот же объём работы.

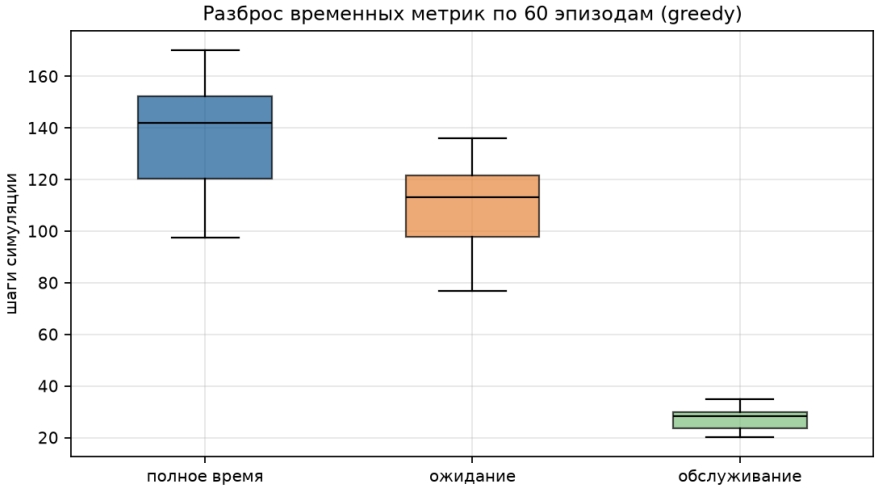


Рис. 5: Разброс временных метрик по 60 эпизодам.

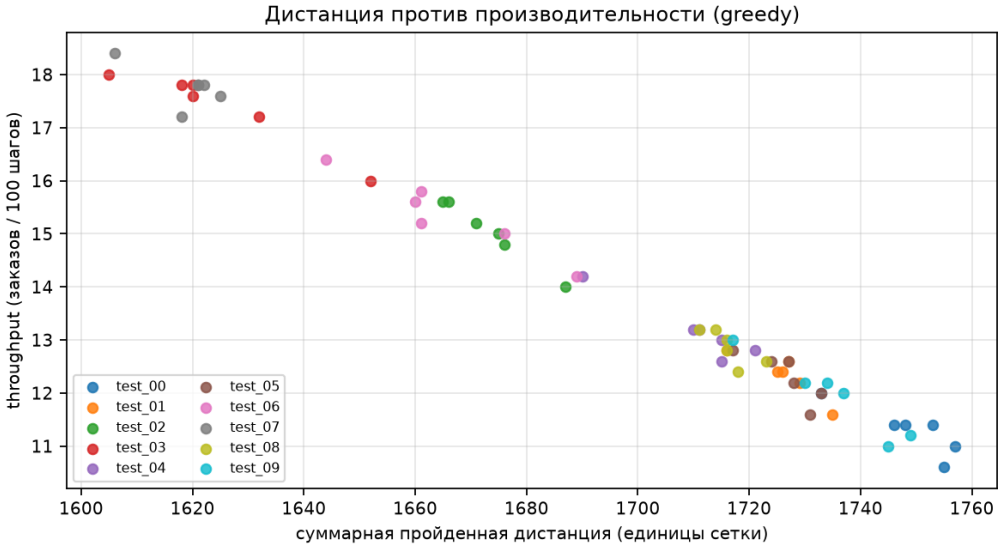


Рис. 6: Суммарная пройденная дистанция против пропускной способности; цвет соответствует тестовой планировке.

Интерпретация

Различия между планировками (пропускная способность от ≈11 до ≈18 заказов/100 шагов) — это *именно* тот сигнал, который должно учитывать исследование обобщения. Обучаясь на 10 планировках и тестируясь на 10 невиданных, RL-политика должна показать, что она не «переобучилась» под конкретную геометрию, а выучила переносимую стратегию распределения.

14 Замечания по коду и наблюдения

Анализ выявил несколько неблокирующих моментов, полезных для развития проекта:

- В `astar()` модуль `heapq` импортируется *внутри* цикла поиска — стоит вынести импорт на уровень модуля (микрооптимизация).
- В `compute_episode_metrics` поле `n_orders_created` вычисляется как `len(sim.orders)`, хотя в симуляции уже ведётся счётчик `n_orders_created`. Оба эквивалентны, поскольку заказы из словаря не удаляются, но дублирование стоит устранить.
- CSV-файл результатов содержит колонку `orders_per_completed`, которой *нет* в текущем dataclass `EpisodeMetrics`. Похоже, файл создан более ранней версией схемы метрик — стоит перегенерировать артефакты либо добавить миграцию.
- `results/**` исключён в `.gitignore` (кроме `.gitkeep`), однако в репозитории присутствуют закоммиченные примеры (`greedy_episodes.csv`, снимки, артефакты greedy). Стоит унифицировать политику хранения артефактов.
- `RewardConfig` пока не используется — это заготовка под Gymnasium-обёртку (веха M3).
- Такие же заглушки `ppo.py`/`dqn.py` явно и честно сигнализируют о незавершённости через `NotImplementedError`.

15 Ограничения, допущения и дорожная карта

15.1 Сознательные упрощения

- Роботы *не избегают* столкновений (клетки могут совпадать). Бесконфликтное движение — будущая работа; на исследовательский вопрос о *распределении* оно не влияет.
- Эпизод имеет *фиксированный горизонт* (`max_steps`); `done == truncated`.
- `timeout_steps` истекает только для *неназначенных* заказов.
- Жадный базлайн детерминирован и *не* настраивается.

15.2 Репродуктивность и научная честность

Число роботов, размеры склада, число планировок и все сиды задаются конфигурацией. Обучающие/тестовые планировки берутся из *непересекающихся* потоков сидов. Один и тот же тестовый набор и сиды используются для всех сравниваемых алгоритмов. Снимки конфигураций и сырые CSV сохраняются автоматически. Никаких «прошитых» результатов нет — оценка каждый раз прогоняет симулятор с нуля.

15.3 Дорожная карта

Веха	Содержание	Статус
M1	Сетка, роботы, поток заказов, A*, рендеринг	Готово
M2	Greedy-базлайн, проверка завершения заказов, метрики	Готово
M3	Обёртка Gymnasium (observation/action/reward)	Следующая
M4	Обучение PPO на одной планировке, save/reload, оценка	Ожидается
M5	Обучение на N планировках $\rightarrow$ оценка на невиданных	Ожидается
M6	PPO vs greedy (+ DQN при практичности) по размеру набора	Ожидается

Запланированы три эксперимента: A (обучение на 10, тест на невиданных), B (размер обучающего набора $1/3/5/10$ на фиксированном тесте), C (greedy vs PPO vs опционально DQN).

16 Установка и запуск

Требуется Python $\geq$ 3.10. Рекомендуется `uv`:

```
uv venv --python 3.14 .venv
uv pip install -e ".[dev,analysis]"   # ядро + pytest + pandas + matplotlib
uv pip install -e ".[rl]"             # стек RL (PyTorch, SB3, gymnasium) — тяжёлый
```

Extra	Зависимости	Назначение
(ядро)	numpy, pyyaml	симулятор + greedy (M1–M2)
`rl`	gymnasium, torch, stable-baselines3	обучение RL (M4–M6)
`analysis`	pandas, matplotlib	анализ и графики
`dev`	pytest	тесты

17 Краткий API-справочник

Символ	Описание
`load_config(path, config_cls, overrides)`	Загрузка YAML в dataclass.
`save_config(config, path)`	Сохранение снимка конфигурации.
`Layout.read / write / is_connected`	Чтение, запись и проверка связности планировки.
`generate_layout / generate_layout_suite`	Генерация одной планировки / набора.
`generate_train_test_suites`	Генерация и сохранение train/test.
`OrderGenerator.maybe_generate(step, pending_count)`	Поток заказов.
`WarehouseSimulation(layout, config, seed)`	Создание симуляции.
`sim.step / sim.run / sim.reset`	Управление эпизодом.
`sim.assign / can_assign`	Валидация и назначение заказа роботу.
`sim.ascii_render`	ASCII-снимок состояния.
`astar / DistanceOracle`	Планирование пути и кэш расстояний.
`GreedyPolicy.assign / make_policy`	Политика и фабрика.
`evaluate_policy / evaluate_from_config`	Оценка политики.
`compare_algorithms / summarize_records`	Сравнение и сводка.
`train / train_from_config`	Раннер обучения.
`compute_episode_metrics / save_records_csv`	Метрики и экспорт.

Итог. Репозиторий `RRS` — это аккуратно структурированный исследовательский фреймворк на вехах M1–M2: полностью реализованы и покрыты тестами симулятор, навигация A*, генерация планировок, жадный базлайн, конфигурация и метрики. RL-часть (M3–M6) корректно оставлена в виде явных заглушек с описанным контрактом интерфейса. Архитектура сознательно разделяет *распределение задач* (изучается RL) и *навигацию* (классический A*), что и позволяет строго изучать обобщение политик на невиданные планировки.